H1N1 Outbreak Model: An 11-Compartment ODE System

With a 9-Day Infectious Period

Project Report

Jaathavan Ranjanathan

Moksh Patel

Ontario Tech University

Course: MATH 4020U

Instructor: Mehran Ebrahimi

Date of Submission: April 9, 2026

Contents

1 Problem Statement 2

2 Background Material and Model Setup 2

2.1 Introduction to Differential Equations and the SIR Model 2

2.2 Why an 11-compartment model? 2

2.3 Model assumptions 3

2.4 Runge-Kutta method as numerical background 3

2.5 Brent's method as root-finding background 4

3 Mathematical Formulation of the Model 5

3.1 Differential equation system 5

3.2 Conservation of population 6

3.3 Substituting the numerical values into the equations 6

4 Methodology 7

4.1 Why numerical methods are required 7

4.2 Calibration of the transmission parameter 7

4.3 Numerical solution procedure 7

4.4 Python implementation used in the methodology 8

5 Results 11

6 Interpretation of the Graphs 11

6.1 Figure 1: Susceptible, total infected, and recovered populations 12

6.2 Figure 2: Infected compartments by day of infection 13

7 Discussion 13

8 Conclusion 14

9 Contributions 14

10 References 14

1 Problem Statement

This project studies the spread of an H1N1-like virus through a closed population of size $N = 1500$. At time $t = 0$, exactly one person is infected and the remaining 1499 people are susceptible. Once a person becomes infected, that person remains infectious for exactly 9 days. After the 9-day infectious period, the person moves into a recovered class and is assumed to remain immune for the rest of the outbreak.

The goal of the project is to build a compartmental differential equation model that reflects this 9-day infectious period, determine the unknown transmission rate β, numerically solve the system, and interpret the resulting outbreak behaviour. In particular, the model must satisfy the condition that after 7 days there are 10 total infected individuals.

2 Background Material and Model Setup

2.1 Introduction to Differential Equations and the SIR Model

At their core, differential equations are mathematical formulas that describe how a system changes over time. In epidemiology, they are essential because they allow us to track the rate at which individuals move between different health states.

The most fundamental framework for this is the SIR model, which categorizes the population into Susceptible (S), Infected (I), and Recovered (R) compartments. The primary advantage of the standard SIR model is its simplicity; it is mathematically elegant and effectively captures the general wave-like dynamics of an epidemic. However, a major drawback is its underlying assumption that the time a person spends infected follows an exponential distribution. Mathematically, this allows for a fraction of the population to recover almost instantly, while others might remain infected for an unrealistically long time.

Alternative modeling approaches exist to handle more complex dynamics. For instance, the SEIR model introduces an "Exposed" compartment for individuals who have contracted the virus but are not yet contagious, allowing for an incubation period. Another alternative is Agent-Based Modeling (ABM), which tracks every single individual as a distinct, programmable node rather than grouping them into continuous, macroscopic compartments. While ABM provides high granularity and allows for tracking individual interactions, it is computationally expensive. We opted for an extended compartmental approach because it provides clear, continuous macroscopic trends and remains highly computationally efficient.

2.2 Why an 11-compartment model?

To resolve the shortcoming of the standard SIR model's exponentially distributed recovery time, we modified the approach to enforce a strict time delay. In this project, the infectious period is known precisely as 9 days.

To represent that structure, the infected population is split into 9 separate stages,

$$I_1(t), I_2(t), \ldots, I_9(t),$$

where $I_j(t)$ denotes the number of people on day j of infection at time t. This leads to a total of 11 compartments:

$$S(t),\ I_1(t), I_2(t), \ldots, I_9(t),\ R(t).$$

This stage-structured formulation explicitly tracks how individuals progress through infection before recovering.

2.3 Model assumptions

The model is based on the following assumptions:

1. The population is closed, so there are no births, deaths, immigration, or emigration.
2. Susceptible individuals become infected through contact with currently infected individuals.
3. Transmission follows mass-action incidence.
4. Each infected individual spends exactly one day in each infected class $I_1, I_2, \ldots, I_9$.
5. After completing the ninth day of infection, the individual enters the recovered class.
6. Recovered individuals remain immune throughout the outbreak.

The total infected population at time t is therefore

$$I(t) = \sum_{j=1}^{9} I_j(t).$$

The force of infection uses the usual mass-action term

$$\frac{\beta}{N} S(t) I(t),$$

where β is the transmission parameter to be determined.

2.4 Runge-Kutta method as numerical background

Because the outbreak model is a coupled nonlinear system—driven primarily by the $S(t) \times I(t)$ multiplication term—it does not have a simple algebraic, closed-form solution. A standard numerical background method for first-order ODE systems is the Runge-Kutta method.

For a general system

$$\mathbf{y}'(t) = \mathbf{f}(t, \mathbf{y}), \qquad \mathbf{y}(t_n) = \mathbf{y}_n,$$

the classical fourth-order Runge-Kutta method advances the solution from t_n to $t_{n+1} = t_n + h$ by computing four intermediate slopes:

$$\begin{aligned}
k_1 &= \mathbf{f}(t_n, \mathbf{y}_n), \\
k_2 &= \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}k_1\right), \\
k_3 &= \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}k_2\right), \\
k_4 &= \mathbf{f}(t_n + h, \mathbf{y}_n + hk_3).
\end{aligned}$$

The next numerical value is then approximated by evaluating the weighted average of these slopes:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right).$$

In this project, the unknown function is the 11-component state vector

$$\mathbf{y}(t) = \left(S(t), I_1(t), I_2(t), \ldots, I_9(t), R(t)\right)^T.$$

In the actual computation, the Python solver `solve_ivp` was used with method `RK45`. We chose RK45 because it is an adaptive method; it automatically adjusts its step size to maintain high precision without wasting computing power.

We favoured RK45 over alternatives like Euler's Method because Euler's Method relies solely on the current slope to draw a straight line to the next point, requiring microscopic step sizes to prevent massive accumulating errors. We also considered implicit solvers like the Backward Differentiation Formula (BDF). While BDF is excellent for highly "stiff" differential equations, our model is well-behaved, making the implicit root-finding per step in BDF unnecessary as it would be overkill.

A known shortcoming of explicit Runge-Kutta methods like RK45 is that they can be computationally heavier per step than simpler methods and may require impractically small time steps if a system suddenly becomes stiff. However, for the parameters in our 11-compartment H1N1 model, RK45 maintains excellent stability and efficiency.

2.5 Brent's method as root-finding background

In addition to solving the ODE system, this project requires calibrating an unknown parameter, β, so that the model satisfies the condition

$$I(7) = 10.$$

Define

$$f(\beta) = I(7; \beta) - 10,$$

where $I(7;\beta)$ is obtained by numerically solving the ODE system with a trial value of β. Calibration is then a scalar root-finding problem:

$$f(\beta) = 0.$$

To solve this equation, we use Brent's method, a robust one-dimensional root-finding algorithm that combines bisection, the secant method, and inverse quadratic interpolation. Brent's method requires a bracket $[\beta_L, \beta_U]$ such that

$$f(\beta_L)\, f(\beta_U) < 0,$$

which guarantees at least one root in the interval (assuming continuity). In our implementation, the bracket $(0.3, 0.4)$ is used after verifying this sign-change condition numerically. Conceptually, RK45 and Brent's method play different roles: RK45 computes $I(7;\beta)$ for any chosen β, while Brent's method updates β iteratively until $f(\beta)$ is driven to zero.

3 Mathematical Formulation of the Model

3.1 Differential equation system

Using the assumptions above, the outbreak is modeled by the following system of ordinary differential equations:

$$\begin{aligned}
\frac{dS}{dt} &= -\frac{\beta}{1500} S\big(I_1 + I_2 + \cdots + I_9\big), \\
\frac{dI_1}{dt} &= \frac{\beta}{1500} S\big(I_1 + I_2 + \cdots + I_9\big) - I_1, \\
\frac{dI_2}{dt} &= I_1 - I_2, \\
\frac{dI_3}{dt} &= I_2 - I_3, \\
\frac{dI_4}{dt} &= I_3 - I_4, \\
\frac{dI_5}{dt} &= I_4 - I_5, \\
\frac{dI_6}{dt} &= I_5 - I_6, \\
\frac{dI_7}{dt} &= I_6 - I_7, \\
\frac{dI_8}{dt} &= I_7 - I_8, \\
\frac{dI_9}{dt} &= I_8 - I_9, \\
\frac{dR}{dt} &= I_9.
\end{aligned}$$

The initial conditions are

$$S(0) = 1499, \qquad I_1(0) = 1, \qquad I_2(0) = \cdots = I_9(0) = 0, \qquad R(0) = 0.$$

3.2 Conservation of population

A useful check on the model is that the total population remains constant. If we add all 11 equations together, then every internal transfer cancels and we obtain

$$\frac{d}{dt}\left(S + I_1 + I_2 + \cdots + I_9 + R\right) = 0.$$

Therefore,

$$S(t) + \sum_{j=1}^{9} I_j(t) + R(t) = 1500$$

for all time t. This confirms that the model conserves population.

3.3 Substituting the numerical values into the equations

From the numerical calibration step, the transmission parameter is found to be

$$\beta \approx 0.339976 \text{ day}^{-1}.$$

Since the population size is $N = 1500$, the transmission coefficient in the mass-action term becomes

$$\frac{\beta}{N} = \frac{0.339976}{1500} \approx 0.0002266507.$$

Thus the first two equations may be written numerically as

$$\begin{aligned}
\frac{dS}{dt} &= -0.0002266507\, S\big(I_1 + I_2 + \cdots + I_9\big), \\
\frac{dI_1}{dt} &= 0.0002266507\, S\big(I_1 + I_2 + \cdots + I_9\big) - I_1.
\end{aligned}$$

The remaining equations are unchanged:

$$\begin{aligned}
\frac{dI_2}{dt} &= I_1 - I_2, & \frac{dI_3}{dt} &= I_2 - I_3, & \frac{dI_4}{dt} &= I_3 - I_4, \\
\frac{dI_5}{dt} &= I_4 - I_5, & \frac{dI_6}{dt} &= I_5 - I_6, & \frac{dI_7}{dt} &= I_6 - I_7, \\
\frac{dI_8}{dt} &= I_7 - I_8, & \frac{dI_9}{dt} &= I_8 - I_9, & \frac{dR}{dt} &= I_9.
\end{aligned}$$

At the initial time $t = 0$, these substitutions give

$$I(0) = I_1(0) + \cdots + I_9(0) = 1,$$

so the initial infection rate is

$$\frac{\beta}{1500}S(0)I(0) = \frac{0.339976}{1500}(1499)(1) \approx 0.33975.$$

Therefore,

$$\left.\frac{dS}{dt}\right|_{t=0} \approx -0.33975, \qquad \left.\frac{dI_1}{dt}\right|_{t=0} \approx 0.33975 - 1 = -0.66025.$$

This means that initially the susceptible class begins decreasing slowly, while the first infected class decreases because individuals are simultaneously moving from day 1 to day 2 of infection.

4 Methodology

4.1 Why numerical methods are required

The model is nonlinear because the infection term involves the product $S(t)I(t)$. Because of this nonlinearity and because the infected population is split across 9 coupled stages, there is no simple closed-form solution for the full system. For that reason, the system must be solved numerically. The Runge-Kutta background summarized in the previous section provides the numerical framework used to advance the solution in time.

4.2 Calibration of the transmission parameter

The unknown parameter β is chosen so that the model satisfies the project condition

$$I(7) = \sum_{j=1}^{9} I_j(7) = 10.$$

To do this, we define a function

$$f(\beta) = I(7;\beta) - 10,$$

where $I(7;\beta)$ is the total infected population at day 7 obtained by numerically solving the system with a trial value of β. The desired value of β is the root of this equation:

$$f(\beta) = 0.$$

In computation, Brent's bracketing method is used with an interval $(0.3, 0.4)$ chosen so that $f(0.3)$ and $f(0.4)$ have opposite signs. Using numerical root-finding together with repeated ODE simulation, we obtain

$$\beta \approx 0.339976.$$

4.3 Numerical solution procedure

The main computational steps were:

1. Define the 11-compartment ODE system.
2. Set the initial conditions $S(0) = 1499$, $I_1(0) = 1$, $I_2(0) = \cdots = I_9(0) = 0$, and $R(0) = 0$.
3. Use a numerical ODE solver based on Runge-Kutta methods to simulate the system over time.
4. Use a root-finding algorithm to adjust β until the condition $I(7) = 10$ is met.
5. Plot the total susceptible, infected, and recovered populations.
6. Plot each infected stage $I_1, \ldots, I_9$ separately to visualize progression through the 9-day infectious period.

4.4 Python implementation used in the methodology

The following Python workflow was used in the numerical part of the project:

- `numpy` for numerical arrays,
- `scipy.integrate.solve_ivp` for solving the ODE system,
- `scipy.optimize.brentq` for determining β,
- `matplotlib` for producing the figures.

A short implementation excerpt is shown below.

Listing 1: Full Python code used to calibrate and solve the H1N1 outbreak model.

```
import numpy as np
from scipy.integrate import solve_ivp
from scipy.optimize import brentq
import matplotlib.pyplot as plt

# --- Constants ---
N = 1500
GAMMA = 1.0  # day^-1 (1 day per infection stage; 9 stages => ~9 days
   infectious)
TARGET_I7 = 10.0
FIT_INTERVAL = (0.3, 0.4)
T_END = 200
N_STAGES = 9
STATE_SIZE = N_STAGES + 2

# --- ODE system: y = [S, I1, I2, ..., I9, R] ---
def rhs(t, y, beta):
    S = y[0]
```

```
    I = y[1:1 + N_STAGES]     # I1..I9
    Itot = I.sum()

    dS = -(beta / N) * S * Itot

    dI = np.zeros(N_STAGES)
    dI[0] = (beta / N) * S * Itot - GAMMA * I[0]  # I1
    for j in range(1, N_STAGES):                   # I2..I9
        dI[j] = GAMMA * I[j-1] - GAMMA * I[j]

    dR = GAMMA * I[-1]  # leaves I9 into R

    return np.concatenate([[dS], dI, [dR]])

# --- Initial conditions ---
y0 = np.zeros(STATE_SIZE)
y0[0] = N - 1   # S(0)
y0[1] = 1       # I1(0)
# I2..I9(0)=0, R(0)=0 already

# --- Helper: total infected at t=7 for a given beta ---
def total_infected_at_7(beta):
    sol = solve_ivp(lambda t, y: rhs(t, y, beta),
                    t_span=(0, 7),
                    y0=y0,
                    t_eval=[7],
                    rtol=1e-9,
                    atol=1e-12)
    return sol.y[1:1 + N_STAGES, 0].sum()

def fit_beta(target_i7=TARGET_I7, interval=FIT_INTERVAL):
    f = lambda beta: total_infected_at_7(beta) - target_i7
    return brentq(f, interval[0], interval[1])

def solve_outbreak(beta, t_end=T_END):
    t_eval = np.linspace(0, t_end, 2001)
    sol = solve_ivp(lambda t, y: rhs(t, y, beta),
                    t_span=(0, t_end),
                    y0=y0,
                    t_eval=t_eval,
                    rtol=1e-9,
                    atol=1e-12)
    return sol

def save_plots(t, S, I_total, R, I_stages):
    plt.figure(figsize=(8, 5))
    plt.plot(t, S, label="S(t)", linewidth=2)
```

```
    plt.plot(t, I_total, label="Total infected I(t)", linewidth=2)
    plt.plot(t, R, label="R(t)", linewidth=2)
    plt.xlabel("Time (days)")
    plt.ylabel("People")
    plt.title("H1N1 Model: S(t), I(t), R(t)")
    plt.legend()
    plt.tight_layout()
    plt.savefig("sir_total.png", dpi=300)
    plt.close()

    plt.figure(figsize=(8, 5))
    for j in range(N_STAGES):
        plt.plot(t, I_stages[j], label=f"I{j+1}")
    plt.xlabel("Time (days)")
    plt.ylabel("People")
    plt.title("H1N1 Model: Infected Stages I1(t) to I9(t)")
    plt.legend(ncol=3, fontsize=8)
    plt.tight_layout()
    plt.savefig("infected_stages.png", dpi=300)
    plt.close()

def main():
    beta_star = fit_beta()
    i7 = total_infected_at_7(beta_star)

    sol = solve_outbreak(beta_star)
    t = sol.t
    S = sol.y[0]
    I_stages = sol.y[1:1 + N_STAGES]
    I_total = I_stages.sum(axis=0)
    R = sol.y[-1]

    peak_I = I_total.max()
    t_peak = t[I_total.argmax()]
    final_R = R[-1]
    final_S = S[-1]
    conservation_error = np.max(np.abs(S + I_total + R - N))

    print(f"Fitted beta = {beta_star:.6f} per day")
    print(f"Check: I(7) = {i7:.6f} people (target {TARGET_I7:.1f})")
    print(f"Peak infected ~ {peak_I:.2f} people at t ~ {t_peak:.1f} days
        ")
    print(f"Final recovered ~ {final_R:.2f} people")
    print(f"Final susceptible ~ {final_S:.2f} people")
    print(f"Max conservation error |S+I+R-N| ~ {conservation_error:.2e}"
        )
```

```
    save_plots(t, S, I_total, R, I_stages)

if __name__ == "__main__":
    main()
```

5 Results

The numerical solution gives the following main results:

Quantity	Approximate value
Transmission parameter β	0.339976
Peak total infected population	787.77 people
Time at peak infection	27.1 days
Final susceptible population	83.32 people
Final recovered population	1416.68 people

These values show that the outbreak spreads through most of the population. Only about 83 people remain uninfected by the end of the simulation, while more than 1416 people eventually move into the recovered class.

6 Interpretation of the Graphs

6.1 Figure 1: Susceptible, total infected, and recovered populations

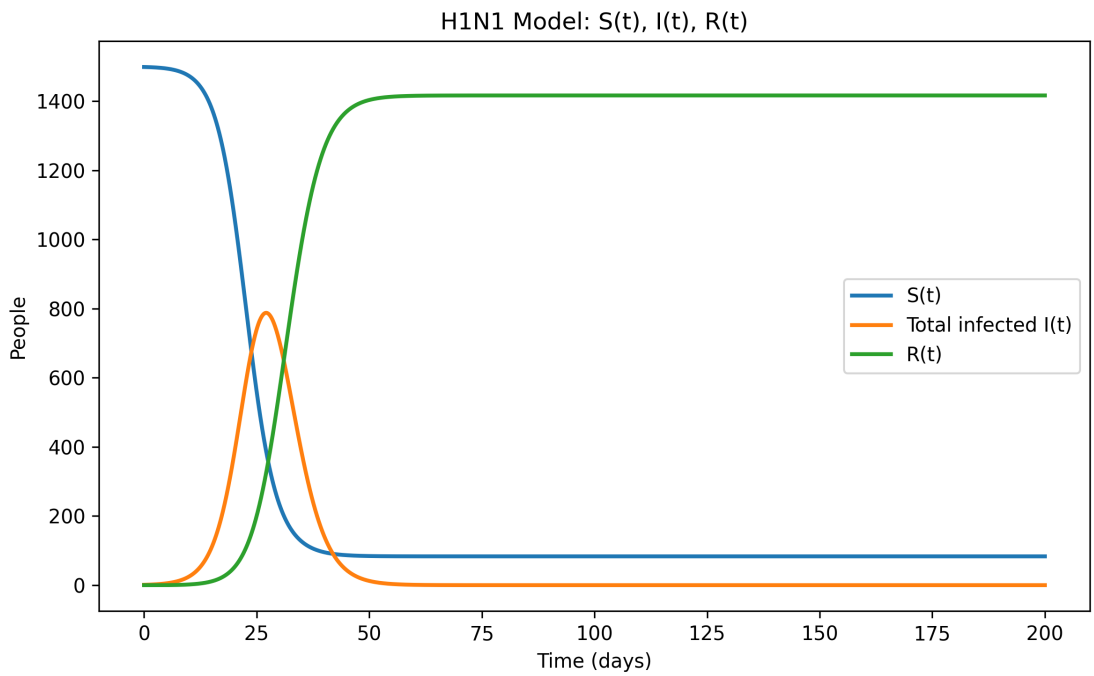

Figure 1: Susceptible, total infected, and recovered populations over time.

Figure 1 shows the overall epidemic trajectory. At the beginning of the outbreak, nearly the entire population is susceptible, so the disease has a large pool of people to infect. This is why the infected curve rises rapidly after the first few days. The blue curve, $S(t)$, starts near 1500 and falls sharply once transmission accelerates. This drop indicates that susceptible individuals are being converted into newly infected individuals at a high rate.

The orange curve, which represents the total infected population

$$I(t) = \sum_{j=1}^{9} I_j(t),$$

first increases, then reaches a maximum of about 788 people around day 27, and then decreases. The decrease after the peak occurs because fewer susceptible individuals remain available to infect, while infected individuals continue progressing through the 9 infectious stages and then recover.

The green curve, $R(t)$, increases steadily over time because recovered individuals accumulate and do not leave the recovered class. By the end of the outbreak, the recovered population levels off near 1417, showing that most of the population experienced infection at some point. Overall, Figure 1 shows the standard pattern of a major epidemic wave: a large susceptible pool at the start, a rapid rise to peak infection, and eventual decline as the susceptible population is depleted.

6.2 Figure 2: Infected compartments by day of infection

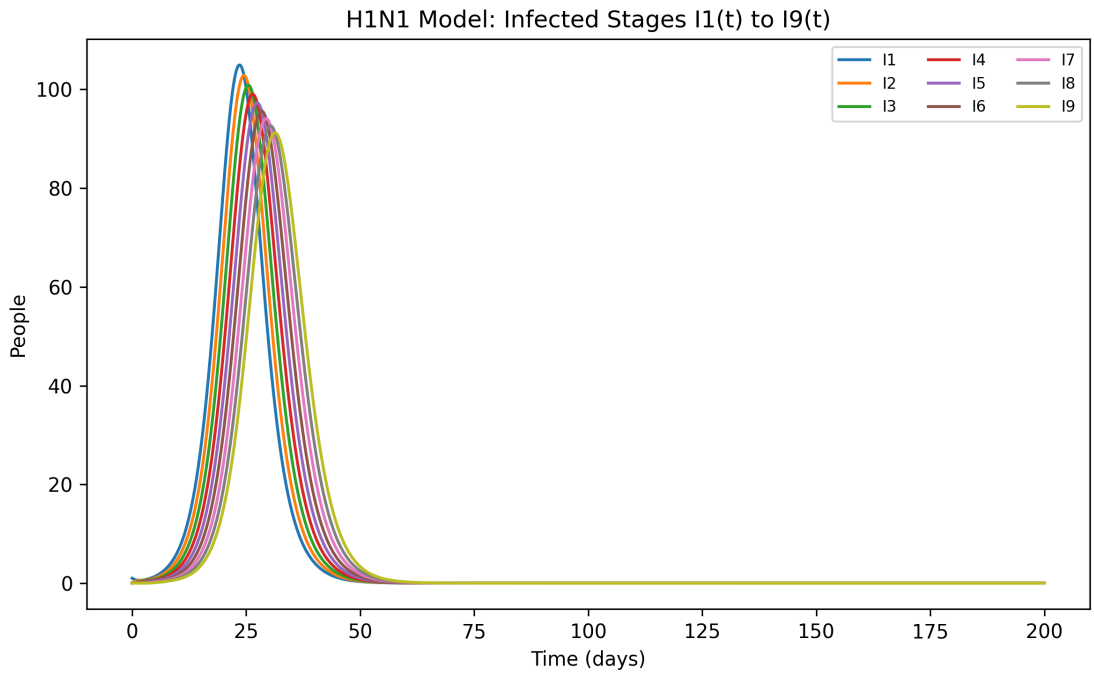

Figure 2: Infected compartments $I_1(t)$ through $I_9(t)$.

Figure 2 breaks the total infected population into the nine infection stages. Each curve corresponds to one day of infection: I_1 is day 1, I_2 is day 2, and so on through I_9. The curves have similar bell-shaped profiles, but they are shifted in time.

The I_1 curve peaks first because newly infected individuals enter the first stage immediately. Then, as those individuals progress through the infectious period, the peaks of $I_2, I_3, \ldots, I_9$ occur one after another. This staggered pattern is exactly what we expect from a model in which individuals remain infectious for a fixed 9-day period and move sequentially through the stages.

The graph also shows that there is a smooth flow of people through the infected compartments. No compartment behaves erratically, which supports the consistency of the numerical solution. The similar peak heights indicate that during the most intense part of the outbreak, large numbers of individuals are present in every stage of infection. In short, Figure 2 confirms that the stage-structured model is working as intended: infection enters through I_1, progresses from one day of infection to the next, and finally exits to the recovered class after the ninth stage.

7 Discussion

The computed value $\beta \approx 0.339976$ produces an outbreak that satisfies the requirement that 10 total individuals are infected after one week. Once that calibration condition is imposed, the model

predicts a large epidemic wave in the closed population.

One reason the outbreak becomes so large is that the initial susceptible population is extremely high relative to the number initially infected. Since almost everyone is susceptible at the beginning, the transmission term

$$\frac{\beta}{1500}SI$$

is initially able to generate many new infections as soon as the infected population begins growing.

The model also demonstrates the value of using multiple infected compartments. If only one infected compartment were used, the model would not explicitly track how long each person has been infected. Here, however, the 9 infected classes give a clearer biological interpretation: every person spends one day in each infection stage before recovery. That structure makes the progression of the disease easier to interpret and better reflects the problem statement.

8 Conclusion

In this project, we constructed and solved an 11-compartment epidemiological model for an H1N1-like outbreak in a closed population of 1500 individuals. The model was designed to reflect a 9-day infectious period by dividing infected individuals into nine separate stages.

Because the resulting system is nonlinear, numerical methods were required. We used an ODE solver together with a root-finding procedure to determine the transmission parameter β so that the model satisfied the condition $I(7) = 10$.

The final calibrated model gave

$$\beta \approx 0.339976,$$

and predicted a substantial outbreak with a peak infected population of approximately 788 people around day 27. By the end of the outbreak, only about 83 susceptible individuals remain, while about 1417 individuals have recovered.

9 Contributions

Moksh Patel: Report, Code, & Presentation (with a focus on the results/discussion)

Jaathavan Ranjanathan: Report, Code, & Presentation (focus on the background/methodology)

10 References

[1] E. Sullivan, *Numerical Methods: An Inquiry-Based Approach With Python*, online textbook, Carroll College. Available online: https://numericalmethodssullivan.github.io/. Last updated: 2022-01-01.

[2] "SEIR model," Wikipedia. https://en.wikipedia.org/wiki/Compartmental_models_in_epidemiology#The_SEIR_model.

[3] "Agent-based model," Wikipedia. https://en.wikipedia.org/wiki/Agent-based_model.

[4] "Runge-Kutta methods," Wikipedia. https://en.wikipedia.org/wiki/Runge%E2%80%93Kutta_methods.

[5] "Euler method," Wikipedia. https://en.wikipedia.org/wiki/Euler_method.

[6] "Brent's method," Wikipedia. https://en.wikipedia.org/wiki/Brent%27s_method.

[7] SciPy Developers, "scipy.integrate.solve_ivp," SciPy API Reference. https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html.

[8] SciPy Developers, "scipy.optimize.brentq," SciPy API Reference. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.brentq.html.

[9] Matplotlib Developers, "Matplotlib Documentation." https://matplotlib.org/stable/contents.html.